

# Resolving Apex Domain CNAME Limitations with Cloudflare, DomainIndia, and Wasmer

A Comprehensive Step-by-Step Resolution Blueprint for Modern Web Apps

## Executive Summary

When hosting modern applications on platforms like Wasmer (which provide dynamic CNAME targets such as `app.id.wasmer.app`), traditional domain registrars like DomainIndia often lack **CNAME flattening** at the root apex level (e.g., `sahelifashion.co.in`). This guidebook outlines the architectural limitation, why traditional DNS fails, and how routing through Cloudflare cleanly solves root forwarding and SSL termination.

## 1. The Core Problem: Apex Records & Registrar Constraints

Standard DNS protocols dictate that a root apex domain (the naked domain without `www`, such as `yourdomain.com`) cannot be a direct CNAME record due to conflicts with mandatory SOA and MX records.

- Wasmer Requirement:** Modern PaaS hosts route traffic via subdomains/CNAME targets rather than static, unchanging IPv4 addresses.
- Registrar Limitation:** Traditional registrars like DomainIndia only support pointing root domains to static A-records (IPv4), causing NXDOMAIN errors when attempting to point directly to dynamic cloud endpoints.
- The User Experience Impact:** Visitors typing `yourdomain.com` hit dead ends or timeout errors, while `www.yourdomain.com` works correctly.

## 2. The Strategic Solution: Cloudflare Integration

To bypass registrar limitations without migrating your actual domain registration away, we introduce **Cloudflare** as the authoritative DNS nameserver provider.

| Component   | Role               | Function                                                                        |
|-------------|--------------------|---------------------------------------------------------------------------------|
| DomainIndia | Registrar          | Holds domain ownership and delegates nameservers to Cloudflare.                 |
| Cloudflare  | DNS & Proxy Engine | Provides native <b>CNAME Flattening</b> at the apex and executes 301 redirects. |
| Wasmer      | Application Host   | Runs the Next.js storefront and processes application logic.                    |

### 3. Step-by-Step Implementation Workflow

---

#### Phase 1: Nameserver Migration at DomainIndia

1. Create a free account on Cloudflare and add your custom domain.
2. Cloudflare scans existing records and outputs two assigned nameservers (e.g., `leanna.ns.cloudflare.com` and `otto.ns.cloudflare.com`).
3. Log into your DomainIndia portal, navigate to domain management, update the nameservers to the Cloudflare values, and save.

#### Phase 2: Configuring DNS & CNAME Flattening

Once Cloudflare reports the domain status as **Active**, configure the DNS records under the Cloudflare dashboard:

- **Subdomain Record:** Add a `CNAME` record with Name `www` pointing to your Wasmer target (`app.id.wasmer.app`), proxied (orange cloud).
- **Apex Record:** Add a `CNAME` record with Name `@` pointing to `www.yourdomain.com`. Cloudflare's CNAME flattening automatically resolves this safely.

#### Phase 3: Setting Up Root Redirect Rules

To ensure traffic hitting the naked root domain smoothly redirects to the secure storefront:

- Navigate to **Rules > Page Rules** in Cloudflare.
- Create a new rule targeting `yourdomain.com/*`.
- Select action **Forwarding URL** with status code **301 - Permanent Redirect**.
- Set destination URL to `https://www.yourdomain.com/$1` (the `$1` wildcard preserves subpaths and deep links).

#### Roll-Back Safety Net

This architecture is entirely non-destructive. If rollback is ever required, simply revert the nameservers in DomainIndia back to their original default values. Your Wasmer application code remains completely untouched throughout the process.